0. **Code:** uppalh8095 Uppalh8095.zip

1. Uppal8095/A. Selection Statements/A. Selection Statements/Program.cs
2. Uppal8095/B. Sentinel Control Loop/B. Sentinel Control Loop/Program.cs
3. Uppal8095/C. Min, Max, Previous Values/C. Min, Max, Previous Values/Program.cs

1. Uppal8095/A. Selection Statements/A. Selection Statements/Program.cs

```
namespace A._Selection_Statements
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int num1;
            Console.WriteLine("Enter number 1 ⟹");
            num1 = int.Parse(Console.ReadLine());
            int num2;
            Console.WriteLine("Enter number 1 ⟹");
            num2 = int.Parse(Console.ReadLine());
            int num3;
            Console.WriteLine("Enter number 1 ⟹");
            num3 = int.Parse(Console.ReadLine());

            int sum = 0;
            decimal average;
            int count = 0;

            int rem1_2 = 0;
            int rem2_3 = 0;
            int rem3_1 = 0;

            if (num1 ≠ 0 && num2 ≠ 0)
            {
                rem1_2 = (num1 / num2 ≠ 0 ? num1 % num2 : num2 % num1);
                if (rem1_2 ≠ 0)
                {
                    sum += rem1_2;
                    count++;
```

A1

num1=0
num2=2
num3=3

A2

num1=0,
num2≠0 is valid

A3

you can not do this.

1. Uppal8095/A. Selection Statements/A. Selection Statements/Program.cs

```
            }

        }
        if (num2 ≠ 0 && num3 ≠ 0)
        {
            rem2_3 = (num2 / num3 ≠ 0 ? num2 % num3 : num3 % num2);
            if (rem2_3 ≠ 0)
            {
                sum += rem2_3;
                count++;
            }

        }
        if (num3 ≠ 0 && num1 ≠ 0)
        {
            rem3_1 = (num3 / num1 ≠ 0 ? num3 % num1 : num1 % num3);
            if (rem3_1 ≠ 0)
            {
                sum += rem3_1;
                count++;

            }
        }

        if (count ≠ 0)
        {
            average = (decimal) sum / count;
            Console.WriteLine($"Average is {sum}/{count} = {average}");
        }
        else
        {
```

1. **Uppal8095/A. Selection Statements/A. Selection Statements/Program.cs**

```
                Console.WriteLine("The average cannot be calculated");
            }

        }
    }
}
```

2. Uppal8095/B. Sentinel Control Loop/B. Sentinel Control Loop/Program.cs

```
namespace B._Sentinel_Control_Loop
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int num1;
            int num2;
            int rem = 0;

            int sum = 0;
            decimal average;
            int count = 0;

            while (true)
            {
                Console.WriteLine($"Enter a pair of integers.");
                num1 = int.Parse(Console.ReadLine());
                num2 = int.Parse(Console.ReadLine());

                if (num1 == 0 || num2 == 0)
                {
                    break;
                }

                rem = (num1 / num2 != 0 ? num1 % num2 : num2 % num1);

                if (rem != 0)
                {
                    sum += rem;
                    count++;
```

B1

B2

B3

2. Uppal8095/B. Sentinel Control Loop/B. Sentinel Control Loop/Program.cs

```
                }


            }
            if (count != 0)
            {
                average = (decimal) sum / count;
                Console.WriteLine($"The average is {sum}/{count} = {average}");
            }
            else
            {
                Console.WriteLine("The average cannot be calculated");
            }

        }
    }
}
```

B4

3. pal8095/C. Min, Max, Previous Values/C. Min, Max, Previous Values/Program.cs ◀

```
namespace C._Min__Max__Previous_Values
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int min = int.MaxValue;
            int max = int.MinValue;
            int count_max = 0;
            int count_min = 0;
            int count = 1;
            int prev;
            int next;
            int num;
            Console.WriteLine("Enter a value:");
            prev = int.Parse(Console.ReadLine());

            while (true)
            {

                Console.WriteLine("enter a value:");
                next = int.Parse(Console.ReadLine());

                if (prev == next)
                {
                    break;
                }

                if (prev > next)
                {
```

C1

previous initially entered ok

C1

C2

check is not needed.

3. pal8095/C. Min, Max, Previous Values/C. Min, Max, Previous Values/Program.cs

```
                if (prev % 10 < next % 10)
                {
                    if (prev > max)
                    {
                        max = prev;
                        count_max = count;

                    }


                }
            }
            if (next > prev)
            {
                if (prev % 10 < next % 10)
                {
                    if (next > max)
                    {
                        max = next;
                        count_max = count;
                    }
                }
            }
            if (prev > next)
            {
                if (prev % 10 > next % 10)
                {
                    if (next < min)
                    {
                        min = next;
                        count_min = count - count_min;
```

C2

C2

Repeated code.

C2

C3

Check is not needed

no need to accumulate.

Why?

3. pal8095/C. Min, Max, Previous Values/C. Min, Max, Previous Values/Program.cs

```
                    }
                }
            }
            if (next > prev)
            {
                if (prev % 10 > next % 10)
                {
                    if (prev < min)
                    {
                        min = prev;
                        count_min = count - count_min;
                    }
                }
            }
            count++;
            prev = next;


        }
        num = (count_max > count_min) ? count_max - count_min :
          count_min - count_max;

        Console.WriteLine($"The number of values entered between the
          max with increasing last digit ({max}) and min with
          decreasing last digit ({min}) is {num}.");



    }
  }
}
```

C3

C3

repeated code

Why : No need to accumulate !

count_min = count;

C4